Uppsala University

Scientific Computing for Data Analysis

1TD352

Mini Project 2

Author:
Group 9:
Arvid Axelsson
Hannes Lundström
Neo Sjöström

March 5, 2026

1 Introduction

The classification of handwritten digits is a common problem in the field of pattern recognition, commonly applied to automated tasks such as reading zip codes on envelopes. In this mini-project we implement a simple and interpretable classifier based on the Singular Value Decomposition (SVD) to distinguish between digits. The process involves two primary datasets: a training dataset of manually classified digits used to develop the algorithm and a test set used to evaluate performance by comparing algorithmic precictions against known labels.

Each digit is represented as a grayscale image where pixel brightness is stored in a matrix. For this analysis, 28×28 pixel images are flattened into vectors of size $m = 784$. By stacking n such vectors for a specific digit d, we form a training matrix $A_d \in \mathbb{R}^{m \times n}$.

The key idea is that images of the same digit (written with different styles) tend to lie approximately in a low-dimensional subspace. By computing the SVD of A_d and keeping the first k left singular vectors, we obtain an orthonormal basis that captures the dominant variations of digit d. A test image is classified as the digit whose subspace gives the smallest reconstruction residual.

In Task 2 we analyze the SVD structure for digits 3 and 8 (singular values and the first three "singular images"). In Task 3 we implement the full classifier, test on all 40 000 test images, and report success rates for $k = 5, \ldots, 15$.

2 Method

Task 2

For digits 3 and 8 we formed two training matrices A_3 and A_8, each of size 784×400. Each column is one handwritten image (flattened 28×28 into a vector in $\mathbb{R}^{784}$), and we used only the first 400 training images for each digit. For each matrix we computed the reduced SVD

$$A_d = U_d \Sigma_d V_d^T, \qquad d \in \{3, 8\},$$

where the diagonal entries of Σ_d are the singular values $\sigma_1 \geq \sigma_2 \geq \cdots$ and the columns of U_d are orthonormal left singular vectors. We plotted the singular

values σ_i versus their index i using linear scales on both axes. To visualize the dominant modes, we also plotted the first three left singular vectors u_1, u_2, u_3 by reshaping each $u_j \in \mathbb{R}^{784}$ into a 28×28 array and displaying it as a grayscale image. These "singular images" describe typical patterns and main variations within the digit class.

Task 3

For each digit $d = 0, \ldots, 9$ we built a training matrix $A_d \in \mathbb{R}^{784\times 400}$ using the first 400 training images of that digit. We then computed the SVD of each A_d and stored the first 15 left singular vectors in an orthonormal basis

$$U_{15}^{(d)} \in \mathbb{R}^{784\times 15}.$$

For a given $k \leq 15$, we define $U_k^{(d)}$ as the first k columns of $U_{15}^{(d)}$.

Given a test image $\delta \in \mathbb{R}^{784}$, we approximate it by projecting it onto each digit subspace. This is done by solving the least-squares problem

$$\min_x \|U_k^{(d)} x - \delta\|_2,$$

which in practice means computing the projection $U_k^{(d)}(U_k^{(d)})^T\delta$. The residual is then computed as

$$r_d(\delta) = \left\| \delta - U_k^{(d)}(U_k^{(d)})^T\delta \right\|_2$$

We classify the test image as the digit with the smallest residual:

$$\hat{d}(\delta) = \arg\min_{d\in\{0,\ldots,9\}} r_d(\delta).$$

In the implementation, we evaluated $k = 5, 6, \ldots, 15$. For each k we computed the residuals for all digits and all test images, selected predictions using `argmin` over the 10 residual vectors, and compared the predicted labels with `TestLabels` to compute success rates. We reported success percentages separately for each digit and plotted the success rate as a function of k.

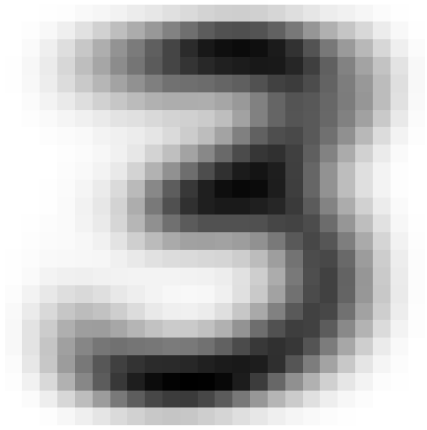
3 Results

3.1 Task 2

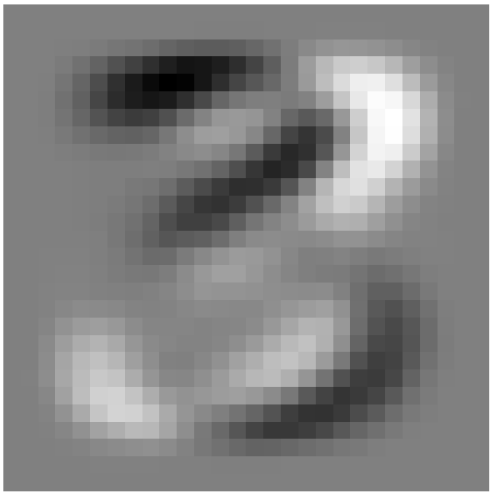
Running the code for Task 2, see appendix A, we get these images of the singular vectors $u1$, $u2$, $u3$. We also get graphs of the singular values for

digit 3 and 8 which both decay rapidly up until approximately σ_{25}. After σ_{25} we can observe that the decrease in value for both digits is much slower and eventually approaches 0.

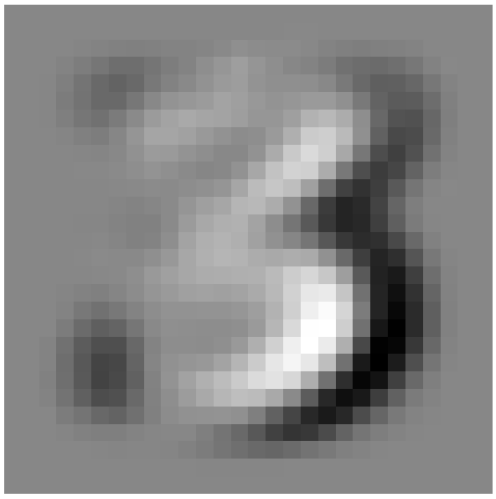
Digit 3: left singular vector u_1



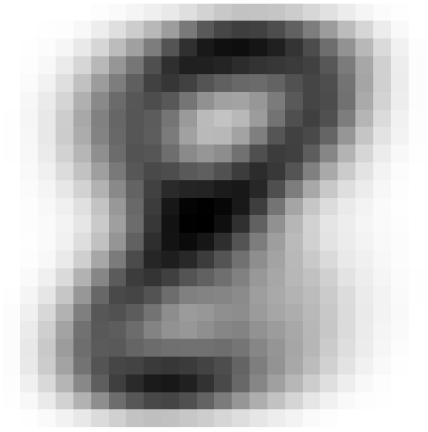
Digit 3: left singular vector u_2



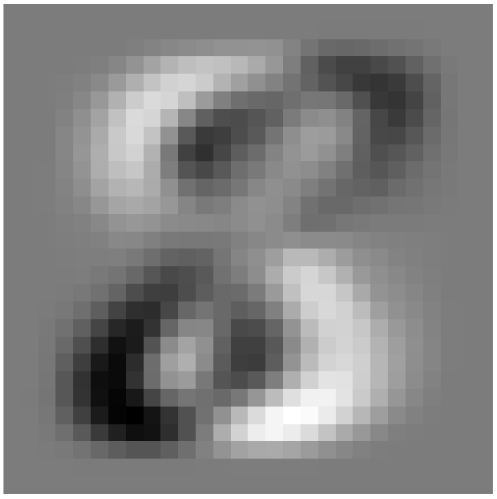
Digit 3: left singular vector u_3



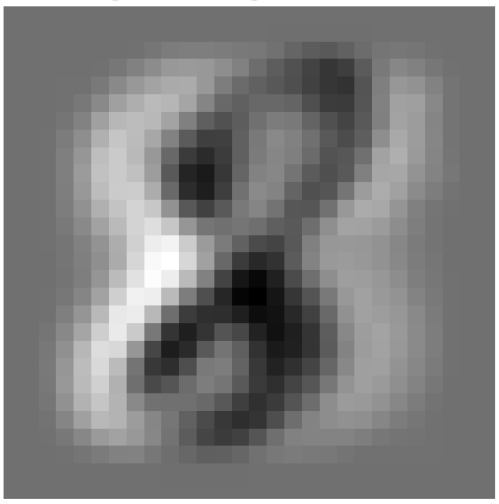
Digit 8: left singular vector u_1

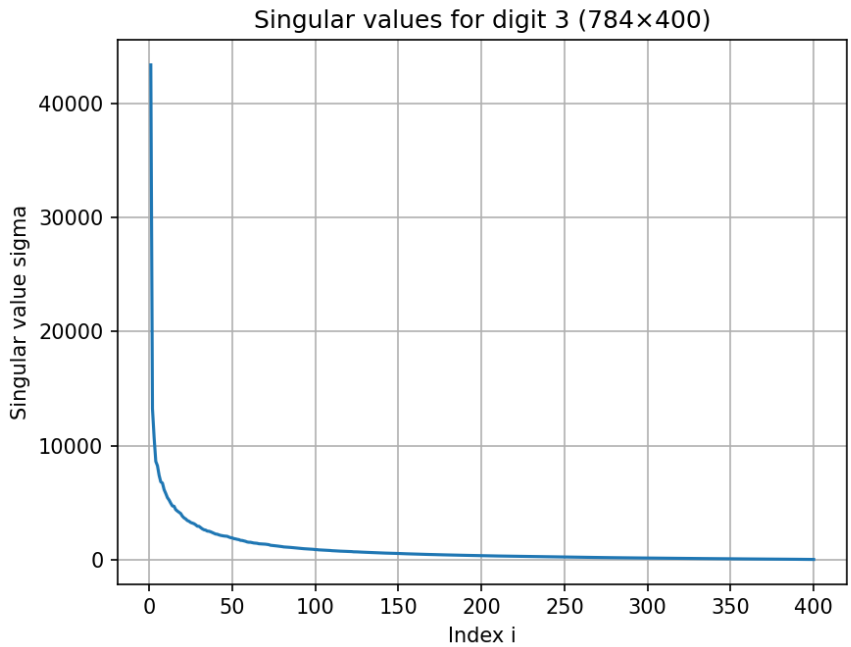


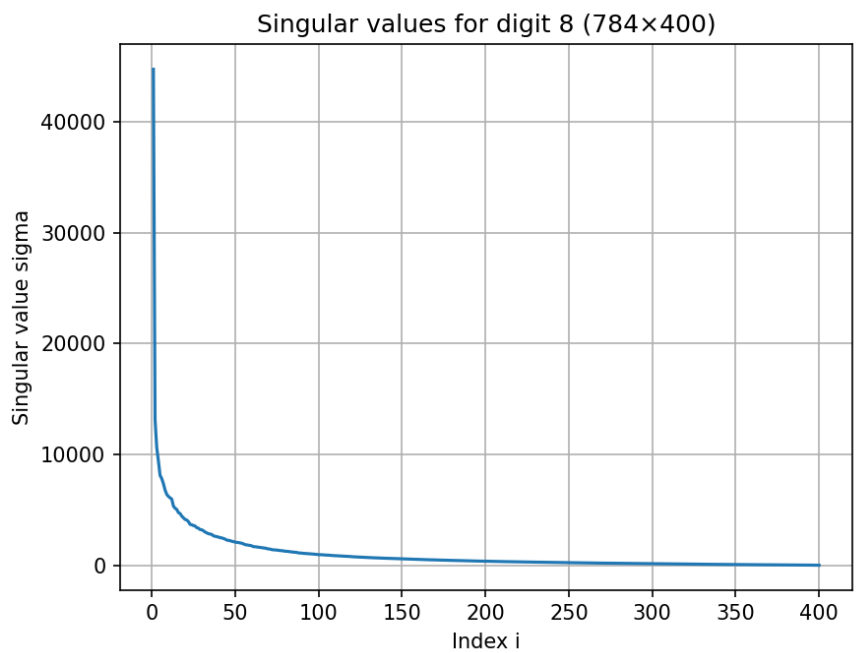
Digit 8: left singular vector u_2



Digit 8: left singular vector u_3

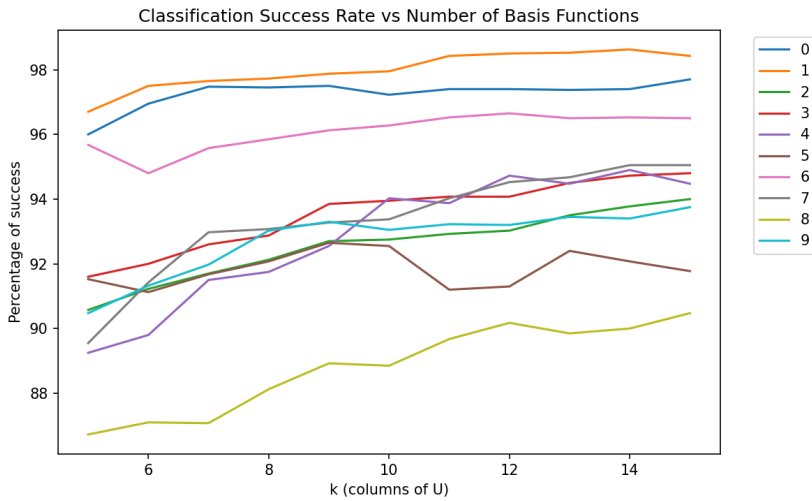



Singular values for digit 3 (784×400)
Singular value sigma
Index i
0
10000
20000
30000
40000
0
50
100
150
200
250
300
350
400


Singular values for digit 8 (784×400)
Singular value sigma
Index i
0
10000
20000
30000
40000
0
50
100
150
200
250
300
350
400

3.2 Task 3

In task 3 we can see that the worst success rate we get is for the digit 8 while the best success rate is for the digit 1. Also success rates generally get better with increase in K.



4 Discussion

The results support the main idea where the images of the same digit can be well approximated by a low-dimensional subspace. In Task 2 this is seen in the rapid decay of the singular values for digits 3 and 8. A fast decay means that most variation between handwriting styles is captured by only a few basis vectors. In both cases the first few singular values are significantly larger than the remaining ones, meaning that a small number of basis vectors explain most of the importance in the data. This justifies using a limited number of singular vectors in the classifier. The first singular image resembles a typical version of the digit, while the next ones describe the most important variations, such as changes in thickness, slant or shape.

In Task 3 the accuracy increases as k grows from 5 to 15, but the improvement becomes smaller for larger k. This behaviour is expected, once the main structure of the digit is captured additional basis vectors mostly describe smaller details and noise. The plot also shows that some digits are easier to classify than others. Digits such as 0 and 1 achieve high accuracy

quickly, while digit 8 is more difficult. This is likely because 8 has greater variation in handwriting and shares visual similarities with other digits, example the number 3.

There are also limitations to the approach. Only 400 training images per digit were used, which may not fully represent all writing styles. In addition, the method relies directly on raw pixel values and is sensitive to shifts, scaling, or small rotations of the digits. Since the model is purely linear, it may not capture more complex nonlinear variations in handwriting.

Overall, the SVD-based classifier performs well despite its simplicity. It clearly demonstrates how low-rank structure can be used for classification and provides an interpretable connection between linear algebra and pattern recognition.

A Python Code Task 2

```python
import numpy as np
import matplotlib.pyplot as plt

#Load training data
TrainMat = np.load('TrainDigits.npy')
TrainLab = np.load('TrainLabels.npy')

#extract the 400 first examples of digit d
def build_a(d, n=400):
    index = (TrainLab == d)
    ad = TrainMat[:, index[0]]
    ad = ad[:, 0:n]
    return ad

#Store each matrix for each digit in a dict
A_dict = {}
U_dict = {}
s_dict = {}

#Create SVD matrix
for d in [3, 8]:
    A_dict[d] = build_a(d, 400).astype(float)
    U, s, Vt = np.linalg.svd(A_dict[d], full_matrices=False)
    U_dict[d] = U
    s_dict[d] = s

#Plot singular values
def plot_singular_values(s, title):
    plt.figure()
    x = np.arange(1, len(s) + 1)
    plt.plot(x, s)
    plt.title(title)
    plt.xlabel("Index i")
    plt.ylabel("Singular value sigma")
    plt.grid(True)
    plt.show()

#Creates plots for digit 3 and 8
plot_singular_values(s_dict[3], "Singular values for digit 3 (784 400 )")
plot_singular_values(s_dict[8], "Singular values for digit 8 (784 400 )")

```

```python
def plot_singular_images(U, digit):
    #Loop for the 3 first images
    for i in range(3):
        ui = U[:, i] #Extract image i from ui matrix
        img = np.reshape(ui, (28, 28)).T
        #Reshape column vector to a 28x28 matrix

        #plot the image of ui matrix
        plt.figure()
        plt.imshow(img, cmap='gray')
        plt.title(f"Digit {digit}: left singular
            vector $u_{i + 1}$")
        plt.axis("off")
        plt.show()

plot_singular_images(U_dict[3], 3)
plot_singular_images(U_dict[8], 8)
```

B Python Code Task 3

```python

#Load the Test Data
TestMat = np.load('TestDigits.npy')
TestLab = np.load('TestLabels.npy').flatten()

#Precompute SVDs for all 10 digits
U_all_dict = {}
max_k = 15

for d in range(10):
    Ad = build_a(d, 400).astype(float)
    U, s, Vt = np.linalg.svd(Ad, full_matrices=False)
    U_all_dict[d] = U[:, :max_k]

k_values = list(range(5, max_k+1))
#Matrix to store the success percentages: 10
#digits x 11 different k values
success_rates = np.zeros((10, len(k_values)))

#Grab the number of test images
num_test_images = TestMat.shape[1]

for i, k in enumerate(k_values):
```

```python
    #Create the residual vector with zeros
    residuals = np.zeros((10, num_test_images))

    #For each digit create the Uk matrix and calculate residuals
    for d in range(10):
        Uk = U_all_dict[d][:, :k]
        projection = Uk @ (Uk.T @ TestMat)
        residual_matrix = TestMat - projection
        residuals[d, :] = np.linalg.norm(residual_matrix, axis=0)

    #Predict the correct digit by choosing the
    #one with the smallest residual
    predictions = np.argmin(residuals, axis=0)

    #Calculate success rate for each digit and
    #store it in our matrix
    for d in range(10):
        true_d_indices = (TestLab == d)
        correct_d = np.sum(predictions[true_d_indices] == d)
        total_d = np.sum(true_d_indices)
        success_rate = (correct_d / total_d) * 100
        success_rates[d, i] = success_rate

plt.figure(figsize=(8, 5))

# Plot a line for each digit
for d in range(10):
    plt.plot(k_values, success_rates[d, :], label=str(d))

plt.xlabel("k (columns of U)")
plt.ylabel("Percentage of success")
plt.title("Classification Success Rate vs Number of Basis Functions")
plt.legend(loc="upper left")
plt.tight_layout()
plt.show()
```